ASSIGNMENT-11.2

2303A51064

B-16

Task 1 — Stack Implementation

Prompt:-

Create a Python Stack class implementing push, pop, peek and isEmpty methods with proper documentation.

Code:-

```python
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add element to stack"""
        self.stack.append(item)

    def pop(self):
        """Remove top element"""
        if not self.is_empty():
            return self.stack.pop()
        return "Stack is empty"

    def peek(self):
        """View top element"""
        if not self.is_empty():
            return self.stack[-1]
        return "Stack is empty"

    def is_empty(self):
        """Check if stack is empty"""
        return len(self.stack) == 0


s = Stack()
s.push(10)
s.push(20)
print("Top element:", s.peek())
print("Removed:", s.pop())
```

```
Top element: 20
Removed: 20
```

Task 2 — Queue Implementation

Prompt:-

Create a Python Queue class implementing enqueue, dequeue, front and size operations.

Code:-

```python
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        return "Queue is empty"

    def front(self):
        if not self.is_empty():
            return self.queue[0]
        return "Queue is empty"

    def size(self):
        return len(self.queue)

    def is_empty(self):
        return len(self.queue) == 0


q = Queue()
q.enqueue(5)
q.enqueue(10)
print("Front:", q.front())
print("Removed:", q.dequeue())
```

```
Front: 5
Removed: 5
```

Task 3 — Singly Linked List

Prompt:-

Write Python code for a singly linked list with node creation, insertion and traversal.

Code:-

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

ll = LinkedList()
ll.insert(10)
ll.insert(20)
ll.insert(30)

ll.display()
```

```
10 -> 20 -> 30 ->
```

Task 4 — Binary Search Tree

Prompt:-

Write Python code for Binary Search Tree with insertion and inorder traversal.

Code:-

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
class BST:
    def insert(self, root, value):
        if root is None:
            return Node(value)
        if value < root.value:
            root.left = self.insert(root.left, value)
        else:
            root.right = self.insert(root.right, value)
        return root
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.value, end=" ")
            self.inorder(root.right)
tree = BST()
root = None
root = tree.insert(root, 50)
tree.insert(root, 30)
tree.insert(root, 70)
tree.insert(root, 20)
tree.insert(root, 40)
print("Inorder Traversal:")
tree.inorder(root)
```

```
Inorder Traversal:
20 30 40 50 70
```

Task 5 — Hash Table

Prompt:-

Implement a hash table in Python using chaining for collision handling with insert, search and delete operations.

Code:-

```python
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_function(self, key):
        return key % self.size
    def insert(self, key):
        index = self.hash_function(key)
        self.table[index].append(key)
    def search(self, key):
        index = self.hash_function(key)
        return key in self.table[index]
    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)
ht = HashTable(10)
ht.insert(15)
ht.insert(25)
print("Search 15:", ht.search(15))
ht.delete(15)
print("Search 15 after delete:", ht.search(15))
```

```
Search 15: True
Search 15 after delete: False
```